



Calibración de Cámaras y Corrección de Distorsión con OpenCV

FACULTAD DE INGENIERÍA

ASIGNATURA:

Computación Visual

PRESENTADO POR:

Gabriela Gallegos Rubio

Juan Esteban Cardenas Huertas

Gabriela Guzman Rivera

Juan David Ardila Diaz

Manuel Eduardo Diaz Sabogal

DOCENTE

Aura Maria Forero Pachon

AÑO
2025

Desarrollo de la práctica	1
1. Toma de fotografías	1
2. Aplicación de distorsión artificial	1
3. Detección del patrón	2
4. Calibración de la cámara	2
5. Corrección de las imágenes	2
Comparaciones visuales	3
Valores obtenidos	3
Sección de preguntas	3

Desarrollo de la práctica

1. Toma de fotografías

Para este paso, tomamos 15 fotografías al tablero de ajedrez desde diferentes distancias, ángulos y rotaciones. En este punto, las imágenes estaban totalmente claras y sin distorsiones.

2. Aplicación de distorsión artificial

En este punto, a las imágenes originales, les aplicamos distorsión óptica simulada por medio de la biblioteca OpenCV. Específicamente, se utilizaron las funciones `cv2.initUndistortRectifyMap` y `cv2.remap`.

Para ello, se construyó la matriz intrínseca y los coeficientes de distorsión de la siguiente forma:

```
Python
# Parámetros de cámara ficticios (matriz intrínseca)
K = np.array([[800, 0, w/2], [0, 800, h/2], [0, 0, 1]], dtype=np.float32)
# Parámetros de distorsión simulada
distorsion1 = np.array([[-0.05], [0.25], [0], [0], [0]], dtype=np.float32)
```

Esto, para poder usar las funciones nombradas anteriormente:

```
Python
# Obtener los mapas de remapeo
map1, map2 = cv2.initUndistortRectifyMap(K, distorsion1, None, K, (w, h),
cv2.CV_32FC1)
# Procesar cada imagen
for i, ruta in enumerate(rutas_imagenes):
    img = cv2.imread(ruta)
    img_distorsionada = cv2.remap(img, map1, map2,
interpolation=cv2.INTER_LINEAR)
    # Guardar imagen distorsionada
    nombre_salida = f"imagenes_distorsionadas/img_dist_{i:02d}.png"
    cv2.imwrite(nombre_salida, img_distorsionada)
```

3. Detección del patrón

En este paso, procesando las imágenes distorsionadas, se buscaba detectar las esquinas del patrón de ajedrez. Para ello, se optimizó la detección con un refinamiento de subpíxeles.

En este paso, nuevamente, se usaron funciones predefinidas de la biblioteca de OpenCV: `cv2.findChessboardCorners` y `cv2.cornerSubPix`.

Todas estas imágenes se guardaron en la carpeta 'detecciones_validas'.

4. Calibración de la cámara

Llegamos al paso clave para el desarrollo de la práctica. En este paso, nuestro objetivo era calibrar la cámara a partir de las fotos con los patrones detectados donde había una distorsión notoria. Para ello usamos la función `cv2.calibrateCamera` y calculamos la matriz de cámara y los coeficientes de distorsión a partir de los puntos detectados.

Finalmente, calculamos el error promedio de reproyección obtenido tras el proceso de calibración.

5. Corrección de las imágenes

Con los parámetros obtenidos anteriormente, buscábamos mitigar la distorsión artificial generada. Para ello, hicimos uso de la función `cv2.undistort`. Para concluir la práctica, guardamos imágenes comparativas entre el proceso completo (distorsión, corrección y original).

Comparaciones visuales

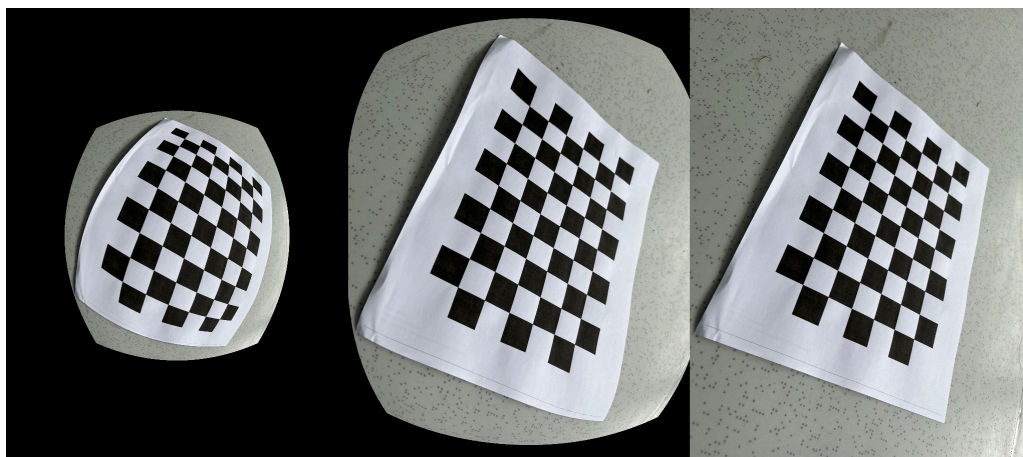


Imagen 1: Comparativa

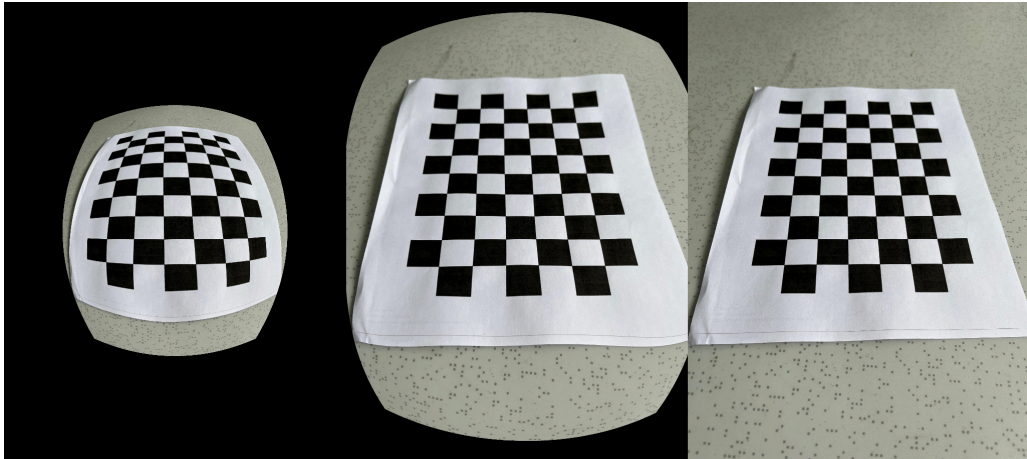


Imagen 2: Comparativa

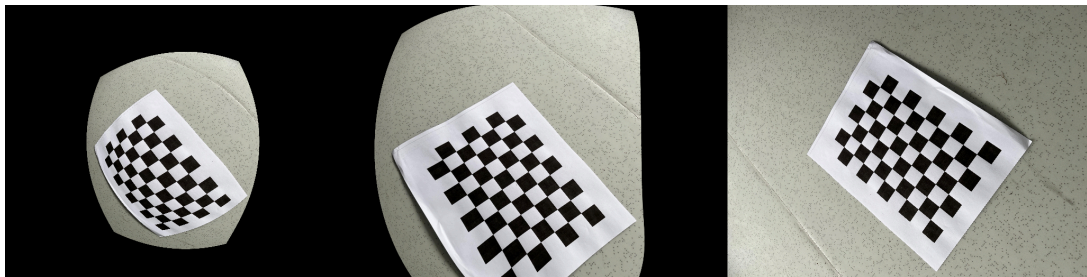


Imagen 3: Comparativa

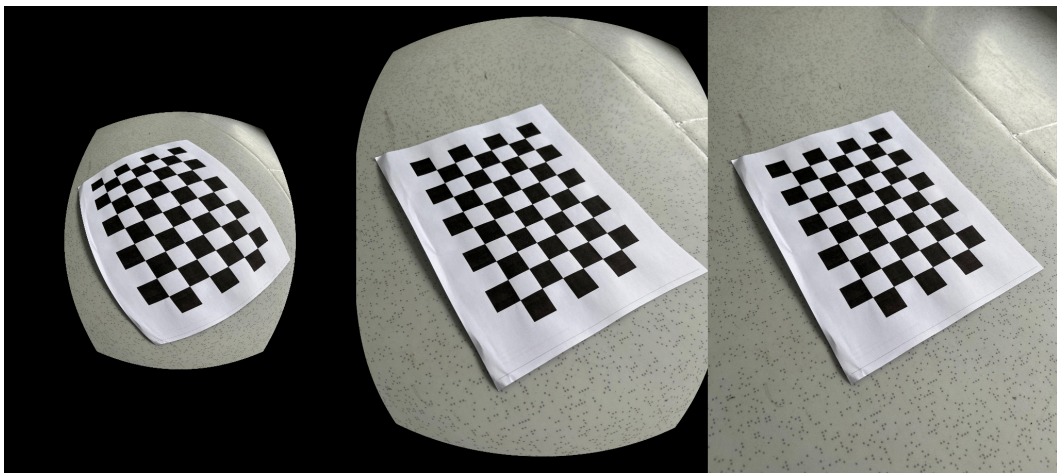
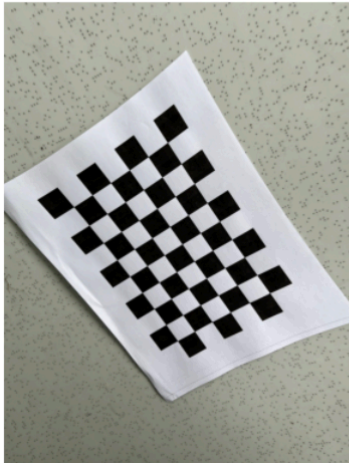


Imagen 4: Comparativa

Comparación: distorsionada_10.jpeg

Distorsionada



Corregida

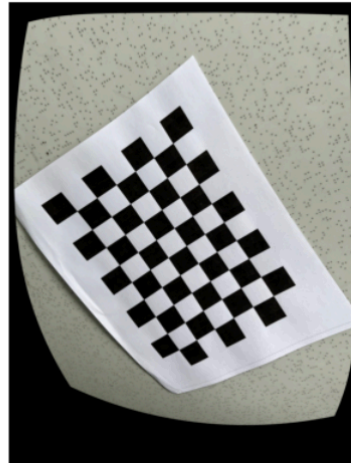
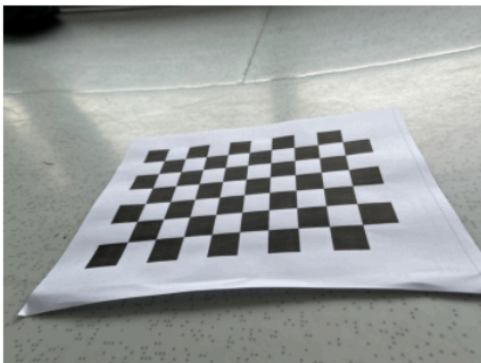


Imagen 5: Comparativa

Comparación: distorsionada_7.jpeg

Distorsionada



Corregida

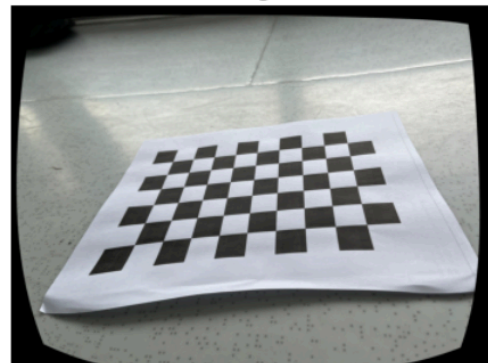


Imagen 6: Comparativa

Valores obtenidos

- **Matriz**

Unset

Matriz de calibración (intrínseca):

```
[[3.48257197e+03 0.00000000e+00 1.52146645e+03]  
 [0.00000000e+00 3.46358213e+03 2.00256612e+03]  
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
```

- **Distorsión**

Unset

```
Coeficientes de distorsión (k1, k2, p1, p2, k3):  
[[-2.85996078 5.38305746 0.00820418 -0.0122413 -1.50464447]]
```

- **Error promedio**

Unset

```
Error medio cuadrático de reproyección: 4.742566821922987
```

Sección de preguntas

1. ¿Cuáles fueron los coeficientes de distorsión obtenidos?
- 2.

Los coeficientes de distorsión obtenidos después del proceso de calibración fueron:

[− 2.85996078, 5.38305746, 0.00820418, − 0.0122413, − 1.50464447]

Estos valores corresponden a los parámetros de distorsión radial y tangencial:

- **k1, k2, k3:** Distorsión radial.
 - **p1, p2:** Distorsión tangencial.
3. ¿Cuántas imágenes fueron necesarias para lograr una calibración aceptable?

Se utilizaron 15 imágenes del patrón de ajedrez tomadas desde diferentes distancias, ángulos y rotaciones. Estas fueron suficientes para obtener una calibración aceptable con un **error medio cuadrático de reproyección de 4.74**, que indica una buena precisión considerando que las imágenes tenían distorsión artificial.

4. ¿Qué diferencias se observan entre las imágenes corregidas y las distorsionadas?

Por un lado, las **imágenes distorsionadas** presentaban deformaciones visibles, como bordes curvados hacia adentro, simulando defectos ópticos típicos de lentes reales. En contraste, las **imágenes corregidas** mostraban líneas rectas y proporciones más realistas, especialmente en las áreas cercanas a los bordes, donde los efectos de la distorsión eran más notables.

En general, la corrección mejoró la geometría de la imagen, alineando mejor los bordes del patrón y eliminando las curvaturas no naturales.

5. ¿Qué aprendiste sobre el uso del patrón de ajedrez y la calibración en visión por computador?

El patrón de ajedrez es una herramienta muy efectiva para la calibración, ya que permite detectar con alta precisión las esquinas en múltiples imágenes. Al utilizar vistas variadas del patrón, es posible estimar con exactitud los parámetros intrínsecos de una cámara y sus coeficientes de distorsión. Este proceso es fundamental en aplicaciones de visión por computadora, ya que mejora considerablemente la precisión en tareas como mediciones, reconstrucción tridimensional y otros algoritmos que dependen de la geometría de la imagen. Además, pude comprobar que incluso las distorsiones ópticas simuladas pueden corregirse eficazmente si se aplica una metodología adecuada utilizando herramientas como OpenCV.